

FNO: Fourier Neural Operator

Mathematical Formulation and PyTorch Reference Implementation

1 Introduction

The Fourier Neural Operator (FNO) is a neural operator architecture that learns mappings between function spaces by parameterizing the kernel of an operator in Fourier space.

Consider a family of parameterized PDEs

$$\mathcal{L}_a(u) = f, \quad (1)$$

where $a(x)$ is an input coefficient or parameter field and $u(x)$ is the corresponding solution. The objective is to learn the solution operator

$$\mathcal{G} : a(\cdot) \mapsto u(\cdot). \quad (2)$$

Unlike a conventional neural network, FNO is designed to approximate the operator itself rather than a mapping tied to one fixed finite-dimensional representation.

The defining feature of FNO is the use of a Fourier integral operator, allowing global interactions to be learned directly in frequency space. The original FNO work demonstrated the approach on Burgers' equation, Darcy flow, and Navier–Stokes equations. [1]

2 Operator Formulation

A general neural-operator layer can be written as

$$v_{t+1}(x) = \sigma \left(W v_t(x) + \int_D \kappa_\phi(x, y) v_t(y) dy \right), \quad (3)$$

where

- $v_t(x)$ is the latent function at layer t ,
- W is a pointwise linear transformation,
- $\kappa_\phi(x, y)$ is a learned integral kernel,
- σ is a nonlinear activation.

FNO parameterizes the integral operator in Fourier space.

3 Fourier Transform

For a function $v : D \rightarrow \mathbb{R}^C$, its Fourier transform is

$$\hat{v}(k) = \int_D v(x) e^{-2\pi i k \cdot x} dx. \quad (4)$$

The inverse transform is

$$v(x) = \int \hat{v}(k) e^{2\pi i k \cdot x} dk. \quad (5)$$

For a discretized field, these operations are replaced by the discrete Fourier transform and its inverse, efficiently computed using the FFT.

4 Fourier Integral Operator

Consider the integral operator

$$(\mathcal{K}v)(x) = \int_D \kappa(x-y)v(y) dy. \quad (6)$$

Because the kernel is translation-invariant, the convolution theorem gives

$$\mathcal{F}[\mathcal{K}v](k) = \hat{\kappa}(k)\hat{v}(k). \quad (7)$$

Therefore,

$$\mathcal{K}v = \mathcal{F}^{-1}[\hat{\kappa}\mathcal{F}[v]]. \quad (8)$$

FNO replaces the fixed Fourier multiplier $\hat{\kappa}(k)$ with a learnable parameterization.

5 Spectral Convolution

Let

$$v_t(x) \in \mathbb{R}^{C_{\text{in}}}. \quad (9)$$

Its Fourier transform is

$$\hat{v}_t(k) = \mathcal{F}[v_t](k). \quad (10)$$

FNO applies a learned complex-valued linear transformation

$$\hat{v}_{t+1}(k) = R_t(k)\hat{v}_t(k), \quad (11)$$

where

$$R_t(k) \in \mathbb{C}^{C_{\text{out}} \times C_{\text{in}}}. \quad (12)$$

The spatial representation is then recovered through

$$v_{t+1}(x) = \mathcal{F}^{-1}[R_t(k)\hat{v}_t(k)]. \quad (13)$$

6 Mode Truncation

A key feature of FNO is that only a finite number of Fourier modes are learned.

Let

$$\mathcal{K} = \{k : |k_i| \leq K_i\} \quad (14)$$

be the retained frequency set.

The spectral convolution becomes

$$(\mathcal{K}_\theta v)(x) = \mathcal{F}^{-1} [\mathbf{1}_{\mathcal{K}}(k) R_\theta(k) \hat{v}(k)]. \quad (15)$$

All frequencies outside $\mathcal{K}$ are discarded.

Thus the number of learned spectral modes controls the resolution of the operator representation.

7 FNO Layer

An FNO layer can be written as

$$\boxed{v_{t+1}(x) = \sigma \left(W_t v_t(x) + \mathcal{F}^{-1} [R_t(k) \mathcal{F}[v_t](k)](x) \right)}. \quad (16)$$

Here,

$$W_t : \mathbb{R}^C \rightarrow \mathbb{R}^C \quad (17)$$

is a pointwise linear transformation.

The spectral term provides global spatial interactions while the pointwise term preserves local channel mixing.

8 Input Lifting

The original input may contain the physical coefficient field and spatial coordinates:

$$z(x) = [a(x), x]. \quad (18)$$

A neural network lifts this representation into a higher-dimensional latent space:

$$v_0(x) = P_{\text{in}}(z(x)). \quad (19)$$

For a linear lifting layer,

$$v_0(x) = W_{\text{in}} z(x) + b_{\text{in}}. \quad (20)$$

9 Stacked FNO Architecture

For L Fourier layers,

$$v_{l+1} = \sigma(W_l v_l + \mathcal{K}_l v_l). \quad (21)$$

Therefore,

$$v_L = \mathcal{FN}\mathcal{O}_\theta(v_0). \quad (22)$$

The final output projection is

$$\hat{u}(x) = P_{\text{out}}(v_L(x)). \quad (23)$$

Thus,

$$\boxed{\mathcal{G}_\theta(a) = P_{\text{out}} \circ \mathcal{FN}\mathcal{O}_\theta \circ P_{\text{in}}(a)}. \quad (24)$$

10 Two-Dimensional FNO

For a two-dimensional domain,

$$x = (x_1, x_2). \quad (25)$$

The Fourier transform is

$$\hat{v}(k_1, k_2) = \int_D v(x_1, x_2) e^{-2\pi i(k_1 x_1 + k_2 x_2)} dx_1 dx_2. \quad (26)$$

The spectral multiplication becomes

$$\hat{v}_{l+1}(k_1, k_2) = R_l(k_1, k_2) \hat{v}_l(k_1, k_2). \quad (27)$$

The inverse transform gives

$$v_{l+1}(x_1, x_2) = \mathcal{F}^{-1} [R_l(k_1, k_2) \hat{v}_l(k_1, k_2)]. \quad (28)$$

For an $H \times W$ grid, the number of retained modes is typically

$$K_1 \times K_2, \quad (29)$$

with

$$K_1 < \frac{H}{2}, \quad K_2 < \frac{W}{2}. \quad (30)$$

11 Why Fourier Space?

Differential operators often have simple representations in Fourier space. For example,

$$\mathcal{F} \left[\frac{\partial u}{\partial x} \right] = 2\pi i k \hat{u}(k), \quad (31)$$

and

$$\mathcal{F} \left[\frac{\partial^2 u}{\partial x^2} \right] = -(2\pi k)^2 \hat{u}(k). \quad (32)$$

Consequently, Fourier representations provide a natural basis for capturing global spatial correlations and differential structure.

12 Discretization

A central property of the neural-operator formulation is that the learned spectral weights are associated with Fourier modes rather than individual grid points.

If

$$v^{(N)} \quad (33)$$

denotes a discretization with N grid points, the same learned spectral parameters can be applied to a different discretization, provided that the retained Fourier modes are represented appropriately.

This gives FNO its characteristic resolution-transfer capability.

13 Computational Complexity

For an N -point discretization, FFT evaluation requires approximately

$$\mathcal{O}(N \log N) \quad (34)$$

operations.

If K Fourier modes are retained and the channel width is C , the spectral multiplication has approximately

$$\mathcal{O}(KC^2) \quad (35)$$

complexity.

Thus an FNO layer has approximate complexity

$$\boxed{\mathcal{O}(N \log N + KC^2)}. \quad (36)$$

This contrasts with a full nonlocal kernel whose direct discretization can require quadratic complexity in the number of spatial points.

14 Training Objective

For a dataset

$$\left\{a^{(i)}, u^{(i)}\right\}_{i=1}^N, \quad (37)$$

the supervised objective can be written as

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \left\| \mathcal{G}_\theta(a^{(i)}) - u^{(i)} \right\|_2^2. \quad (38)$$

A relative L_2 objective is

$$\mathcal{L}_{\text{rel}} = \frac{1}{N} \sum_{i=1}^N \frac{\left\| \mathcal{G}_\theta(a^{(i)}) - u^{(i)} \right\|_2}{\left\| u^{(i)} \right\|_2 + \epsilon}. \quad (39)$$

15 PyTorch Implementation

```

1
2 import torch
3 import torch.nn as nn
4 import torch.fft
5
6
7 class SpectralConv2d(nn.Module):
8
9     def __init__(
10         self,
11         in_channels,
12         out_channels,
13         modes1,
14         modes2
15     ):
16         super().__init__()
```

```

17
18     self.in_channels = in_channels
19     self.out_channels = out_channels
20     self.modes1 = modes1
21     self.modes2 = modes2
22
23     scale = (
24         1.0 /
25         (in_channels * out_channels)
26     ) ** 0.5
27
28     self.weight1 = nn.Parameter(
29         scale * torch.randn(
30             in_channels,
31             out_channels,
32             modes1,
33             modes2,
34             dtype=torch.cfloat
35         )
36     )
37
38     self.weight2 = nn.Parameter(
39         scale * torch.randn(
40             in_channels,
41             out_channels,
42             modes1,
43             modes2,
44             dtype=torch.cfloat
45         )
46     )
47
48     def compl_mul2d(
49         self,
50         x,
51         weight
52     ):
53
54         return torch.einsum(
55             "bixy,ioxy->boxy",
56             x,
57             weight
58         )
59
60     def forward(self, x):
61
62         batch_size = x.shape[0]
63
64         x_ft = torch.fft.rfft2(
65             x
66         )
67
68         out_ft = torch.zeros(
69             batch_size,
70             self.out_channels,
71             x.size(-2),
72             x.size(-1) // 2 + 1,
73             dtype=torch.cfloat,
74             device=x.device

```

```

75         )
76
77         out_ft[
78             :,
79             :,
80             :self.modes1,
81             :self.modes2
82         ] = self.compl_mul2d(
83             x_ft[
84                 :,
85                 :,
86                 :self.modes1,
87                 :self.modes2
88             ],
89             self.weight1
90         )
91
92         out_ft[
93             :,
94             :,
95             -self.modes1:,
96             :self.modes2
97         ] = self.compl_mul2d(
98             x_ft[
99                 :,
100                :,
101                -self.modes1:,
102                :self.modes2
103             ],
104             self.weight2
105         )
106
107         return torch.fft.irfft2(
108             out_ft,
109             s=x.shape[-2:]
110         )
111
112
113 class FNOBlock2d(nn.Module):
114
115     def __init__(
116         self,
117         width,
118         modes1,
119         modes2
120     ):
121         super().__init__()
122
123         self.spectral = SpectralConv2d(
124             width,
125             width,
126             modes1,
127             modes2
128         )
129
130         self.pointwise = nn.Conv2d(
131             width,
132             width,

```

```

133         kernel_size=1
134     )
135
136     def forward(self, x):
137
138         x1 = self.spectral(x)
139
140         x2 = self.pointwise(x)
141
142         return torch.nn.functional.gelu(
143             x1 + x2
144         )
145
146
147 class FNO2d(nn.Module):
148
149     def __init__(
150         self,
151         in_channels,
152         out_channels,
153         width=64,
154         modes1=12,
155         modes2=12,
156         depth=4
157     ):
158         super().__init__()
159
160         self.input_projection = nn.Conv2d(
161             in_channels,
162             width,
163             kernel_size=1
164         )
165
166         self.blocks = nn.ModuleList([
167             FNOBlock2d(
168                 width,
169                 modes1,
170                 modes2
171             )
172             for _ in range(depth)
173         ])
174
175         self.output_projection = nn.Sequential(
176             nn.Conv2d(
177                 width,
178                 width,
179                 kernel_size=1
180             ),
181             nn.GELU(),
182             nn.Conv2d(
183                 width,
184                 out_channels,
185                 kernel_size=1
186             )
187         )
188
189     def forward(self, x):
190

```

```

191         x = self.input_projection(x)
192
193     for block in self.blocks:
194         x = block(x)
195
196     return self.output_projection(x)
197
198
199 if __name__ == "__main__":
200
201     device = (
202         "cuda"
203         if torch.cuda.is_available()
204         else "cpu"
205     )
206
207     batch_size = 8
208     height = 64
209     width = 64
210
211     model = FNO2d(
212         in_channels=3,
213         out_channels=1,
214         width=64,
215         modes1=12,
216         modes2=12,
217         depth=4
218     ).to(device)
219
220     x = torch.randn(
221         batch_size,
222         3,
223         height,
224         width,
225         device=device
226     )
227
228     y = model(x)
229
230     print("Input:", x.shape)
231     print("Output:", y.shape)

```

16 Summary

The defining FNO operation is

$$\mathcal{K}_\theta(v) = \mathcal{F}^{-1} [R_\theta(k) \mathcal{F}(v)(k)]. \quad (40)$$

An FNO layer is

$$v_{l+1} = \sigma(W_l v_l + \mathcal{K}_{\theta_l}(v_l)). \quad (41)$$

The complete operator is therefore

$$a(\cdot) \xrightarrow{\text{lifting}} v_0 \xrightarrow{\text{Fourier layers}} v_L \xrightarrow{\text{projection}} \hat{u}(\cdot). \quad (42)$$

The essential distinction from DeepONet is that FNO represents the operator through learned global Fourier integral transformations, whereas DeepONet uses a branch–trunk separable representation.

References

- [1] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar,
Fourier Neural Operator for Parametric Partial Differential Equations,
arXiv:2010.08895, 2020.
<https://arxiv.org/abs/2010.08895>